Podstawy AI – Sterownik wentylatora w języku FCL

FCL (*Fuzzy Control Language*, norma IEC 61131-7) to standardowy, tekstowy format opisu sterowników rozmytych, używany w przemyśle. Plik FCL zawiera wszystko, co sterownik wie: zmienne, funkcje przynależności, reguły i ustawienia operatorów.

Potrzebne pliki (w tym samym folderze co notebook; w Google Colab: ikona folderu po lewej → „Prześlij"):

- `wentylator.fcl` – sterownik do analizy,
- `fcl_narzedzie.py` – narzędzie, które czyta pliki FCL i liczy wynik.

Nic nie trzeba instalować – narzędzie używa tylko `numpy` i `matplotlib`, które są w Colab.

Plan:

1. Analiza pliku FCL (A–C)
2. Dodanie stanu `very_hot` i nowej reguły
3. Testy sterownika
4. Trzy konfiguracje: implikacja, wyostrzanie, suma zbiorów
5. Ocena sterownika
6. Drugie wejście: wilgotność

Do oddania: pliki FCL (co najmniej 3 konfiguracje i wersja z wilgotnością) oraz ten notebook zapisany jako PDF (Plik → Drukuj → Zapisz jako PDF).

```
from fcl_narzedzie import wynik, tabela, rysuj_zbiory, krzywa, wariant, rozs
print(open("wentylator.fcl", encoding="utf-8").read())
```

```
(* Podstawy AI – sterownik rozmyty wentylatora
   Jedno wejście: temperatura, jedno wyjście: prędkość wentylatora. *)

FUNCTION_BLOCK wentylator

VAR_INPUT
    temperature : REAL;     (* temperatura w pomieszczeniu [°C] *)
END_VAR

VAR_OUTPUT
    fan_speed : REAL;       (* prędkość wentylatora [% mocy] *)
END_VAR

FUZZIFY temperature
    RANGE := (0 .. 40);
    TERM cold := (0, 1) (10, 1) (20, 0);
    TERM warm := (10, 0) (20, 1) (30, 0);
    TERM hot  := (20, 0) (30, 1) (40, 1);
END_FUZZIFY
```

```
DEFUZZIFY fan_speed
    RANGE := (0 .. 100);
    TERM low     := (0, 1) (20, 1) (45, 0);
    TERM medium  := (25, 0) (50, 1) (75, 0);
    TERM high    := (55, 0) (80, 1) (95, 0);
    TERM maximum := (85, 0) (100, 1);
    METHOD : COG;           (* wyostrzanie: środek ciężkości *)
    DEFAULT := 0;           (* wynik, gdy żadna reguła nie działa *)
END_DEFUZZIFY

RULEBLOCK reguly
    AND  : MIN;             (* "i" w części JEŚLI *)
    ACT  : MIN;             (* implikacja: MIN przycina, PROD skaluje *)
    ACCU : MAX;             (* suma zbiorów na wyjściu: MAX, BSUM *)

    RULE 1 : IF temperature IS cold THEN fan_speed IS low;
    RULE 2 : IF temperature IS warm THEN fan_speed IS medium;
    RULE 3 : IF temperature IS hot  THEN fan_speed IS high;
END_RULEBLOCK

END_FUNCTION_BLOCK
```

---

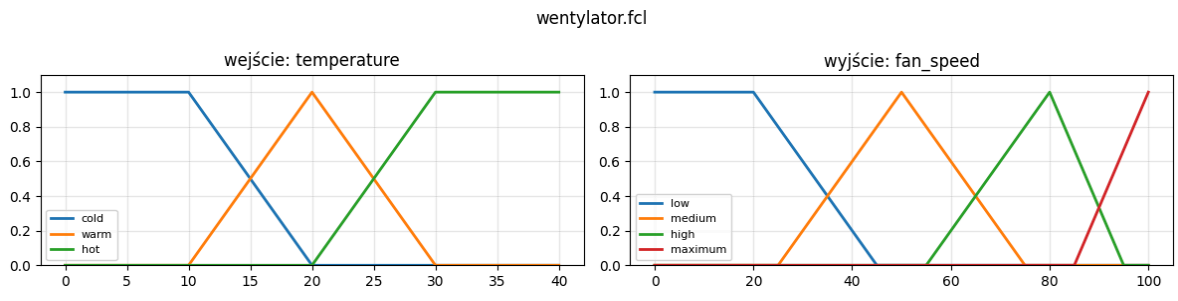
1. Analiza pliku FCL

Plik składa się z bloków. Każdy odpowiada jednemu krokowi sterownika rozmytego:

blok	co zawiera
`VAR_INPUT` / `VAR_OUTPUT`	nazwy wejść i wyjść
`FUZZIFY`	funkcje przynależności wejścia, zapisane punktami `(x, μ)` – łamana przez te punl
`DEFUZZIFY`	funkcje przynależności wyjścia, metoda wyostrzania `METHOD`, wartość `DEFAULT`
`RULEBLOCK`	operatory (`AND`, `ACT` – implikacja, `ACCU` – suma zbiorów) i reguły `IF … THEN …`

Przykład: `TERM cold := (0, 1) (10, 1) (20, 0);` znaczy: do 10 °C „zimno” w stopniu 1, potem stopień maleje liniowo do 0 przy 20 °C.

```
rysuj_zbiory("wentylator.fcl")
```



Zadanie 1A. Wypisz zmienne wejściowe i wyjściowe (nazwa, zakres, jednostka).

Zadanie 1B. Wypisz funkcje przynależności każdej zmiennej i opisz słowami, co znaczą (np. „cold – do 10 °C na pewno zimno, od 20 °C na pewno nie”).

Zadanie 1C. Wypisz reguły i opisz jednym zdaniem, jak działa ten sterownik.

Twoja odpowiedź:

1A. Zmienną wejściową jest temperatura (temperature) w zakresie 0-40°C, zaś zmienną wyjściową jest prędkość wentylatora (fan_speed) w zakresie 0-100%.

1B. Funkcje przynależności dla temperatury to:

"cold" – od 0 do 10°C temperatura jest na pewno zimna. Od 10 do 20°C przynależność maleje do zera. Powyżej 20°C nie jest już zimno.

"warm" – od 10 do 20°C przynależność rośnie. Przy 20°C wynosi 1 (na pewno ciepło). Od 20 do 30°C maleje do zera.

"hot" – poniżej 20°C nie jest gorąco. Od 20 do 30°C przynależność rośnie. Od 30 do 40°C temperatura jest na pewno gorąca.

Funkcje przynależności dla prędkości wentylatora to:

"low" – prędkość wentylatora jest niska. Od 0 do 20% na pewno niska, a następnie przynależność maleje do zera przy około 45%.

"medium" – od 25% do 50% przynależność rośnie. Przy 50% jest maksymalna, a następnie maleje do zera przy 75%.

"high" – od około 55% do 80% przynależność rośnie. Przy 80% jest maksymalna, a następnie maleje do zera przy około 95%.

"maximum" – od około 85% do 100% przynależność rośnie. Przy 100% jest maksymalna.

1C. Reguły sterownika: Jeśli temperatura wskazuje "cold", to prędkość wentylatora ustawiana jest na "low", jeśli temperatura to "warm", prędkość ustawiana jest na "medium", a jeżeli temperatura to "hot", to prędkość wentylatora wynosi "high". Sterownik analizuje temperaturę otoczenia i na podstawie funkcji przynależności określa, czy jest zimno, ciepło czy gorąco. Następnie, korzystając z reguł, dobiera odpowiednią prędkość wentylatora, a więc im wyższa temperatura, tym szybciej pracuje wentylator.

---

2. Dodanie stanu `very_hot`

Komórka poniżej zapisuje plik `wentylator_k1.fcl`. Na start zawiera kopię oryginału. Zmień go bezpośrednio w komórce:

1. **(A)** Dodaj do `temperature` stan `very_hot`. Pamiętaj, że `hot` i `very_hot` muszą się **nakładać**, a `hot` powinien przestać być płaski do 40 °C (inaczej przy 40 °C będzie jednocześnie w pełni „hot" i „very_hot").
2. **(B)** Dodaj regułę: jeśli `very_hot`, to `fan_speed` = `maximum`.

Uruchom komórkę – zapisze plik. Potem uruchom następną, żeby zobaczyć wykres.

```
%%writefile wentylator_k1.fcl
(* TODO: dodaj stan very_hot i regułę dla niego *)

FUNCTION_BLOCK wentylator

VAR_INPUT
    temperature : REAL;     (* temperatura w pomieszczeniu [°C] *)
END_VAR

VAR_OUTPUT
    fan_speed : REAL;       (* prędkość wentylatora [% mocy] *)
END_VAR

FUZZIFY temperature
    RANGE := (0 .. 40);
    TERM cold := (0, 1) (10, 1) (20, 0);
    TERM warm := (10, 0) (20, 1) (30, 0);
    TERM hot  := (20, 0) (30, 1) (40, 0);
    TERM very_hot := (30, 0) (40, 1);
END_FUZZIFY

DEFUZZIFY fan_speed
    RANGE := (0 .. 100);
    TERM low     := (0, 1) (20, 1) (45, 0);
    TERM medium  := (25, 0) (50, 1) (75, 0);
    TERM high    := (55, 0) (80, 1) (95, 0);
    TERM maximum := (85, 0) (100, 1);
    METHOD : COG;           (* wyostrzanie: środek ciężkości *)
    DEFAULT := 0;           (* wynik, gdy żadna reguła nie działa *)
END_DEFUZZIFY

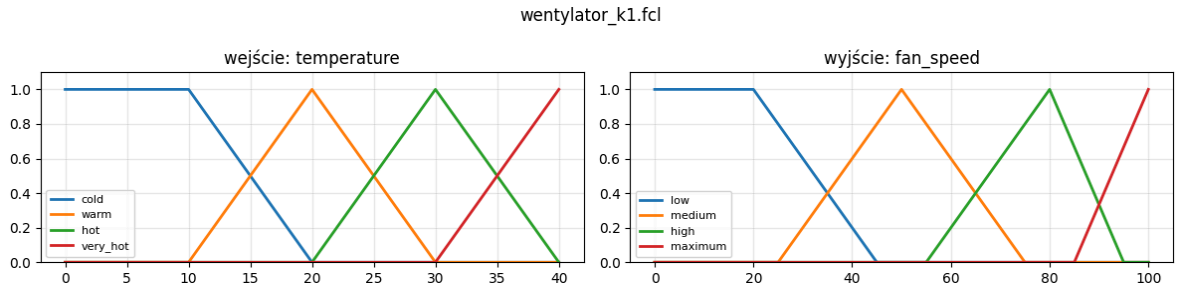
RULEBLOCK reguly
    AND  : MIN;             (* "i" w części JEŚLI *)
    ACT  : MIN;             (* implikacja: MIN przycina, PROD skaluje *)
    ACCU : MAX;             (* suma zbiorów na wyjściu: MAX, BSUM *)

    RULE 1 : IF temperature IS cold THEN fan_speed IS low;
    RULE 2 : IF temperature IS warm THEN fan_speed IS medium;
    RULE 3 : IF temperature IS hot  THEN fan_speed IS high;
    RULE 4 : IF temperature IS very_hot THEN fan_speed IS maximum;
END_RULEBLOCK

END_FUNCTION_BLOCK
```

```
Overwriting wentylator_k1.fcl
```

```
rysuj_zbiory("wentylator_k1.fcl")
```



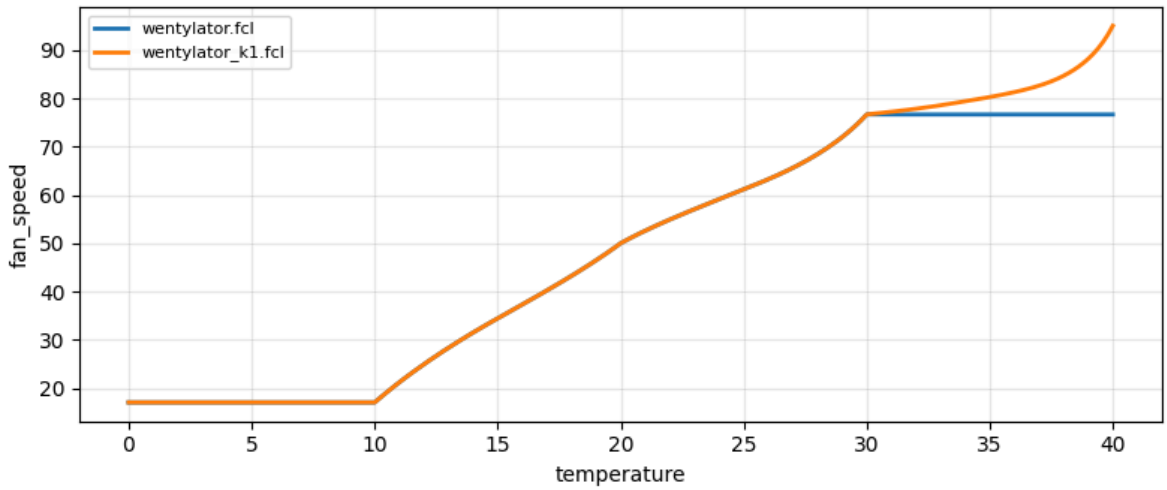
---

3. Testy sterownika

Zadanie 2C. Porównaj oryginał i Twoją wersję dla temperatur od 0 do 40 °C co 5 stopni. Wykres pokazuje prędkość dla wszystkich temperatur naraz.

```
TEMPERATURY = [0, 5, 10, 15, 20, 25, 30, 35, 40]
tabela(["wentylator.fcl", "wentylator_k1.fcl"], "temperature", TEMPERATURY)
krzywa(["wentylator.fcl", "wentylator_k1.fcl"], "temperature")
```

temperature	wentylator	wentylator_k1
0	17.03	17.03
5	17.03	17.03
10	17.03	17.03
15	34.44	34.44
20	50.00	50.00
25	61.15	61.15
30	76.67	76.67
35	76.67	80.27
40	76.67	95.03



Pytanie 3a. Co robi oryginalny sterownik powyżej 30 °C? Co zmieniło dodanie `very_hot`?

Pytanie 3b. Jaką prędkość daje sterownik przy 0 °C, a jaką przy 40 °C? Czy wentylator kiedykolwiek się wyłącza albo osiąga 100%? Dlaczego?

Twoja odpowiedź:

3a. Oryginalny sterownik powyżej 30°C utrzymuje stałą prędkość wentylatora, podczas gdy dzięki dodaniu "very_hot" przyspiesza on dodatkowo pracę wentylatora.

3b. Przy temperaturze 0°C sterownik daje prędkośc około 15%, zaś przy 40°C daje prędkość około 95%. Wentylator nigdy się nie wyłącza, gdyż minimalna wyliczona prędkość to około 15%. Nie osiąga również 100%, ponieważ reguły sterownika i funkcje przynależności powodują wyliczenie wartości pośredniej. Nawet przy bardzo wysokiej temperaturze prędkość nie musi osiągać maksimum.

---

4. Trzy konfiguracje

Zadanie 3A. Przetestuj sterownik z innymi ustawieniami. Każda konfiguracja to osobny plik FCL, w którym zmieniasz **jedną** rzecz:

ustawienie w FCL	co określa	wariant do sprawdzenia
`ACT`	implikacja: jak siła reguły zmienia jej wniosek	`PROD` zamiast `MIN`
`METHOD`	wyostrzanie: jak z kształtu zrobić jedną liczbę	`MM` (środek maksimum) zamiast `COG`
`ACCU`	suma zbiorów na wyjściu: jak łączyć wnioski reguł	`BSUM` (suma ograniczona) zamiast `MAX`

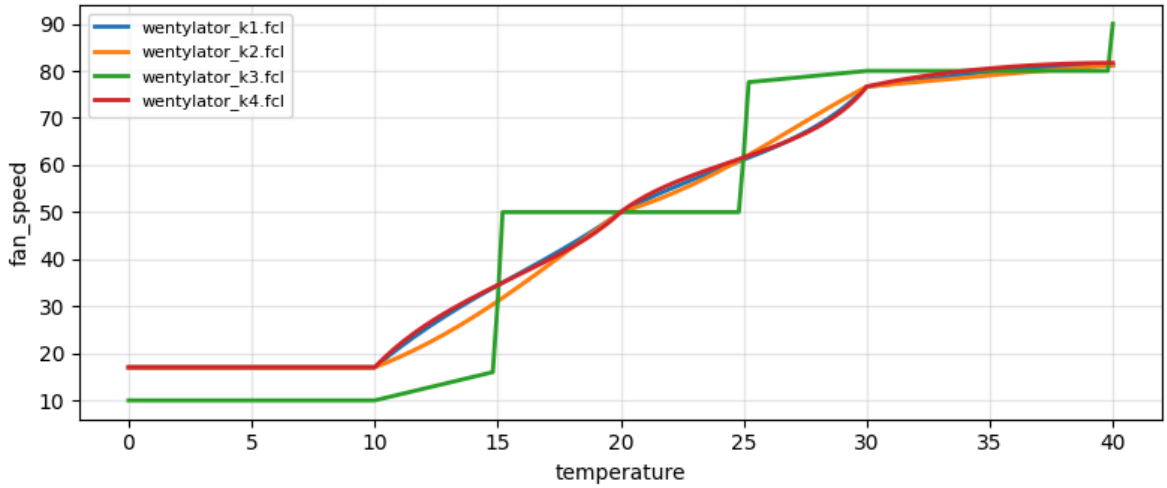
Funkcja `wariant` tworzy kopię pliku ze zmienionym ustawieniem. Możesz też skopiować plik ręcznie i zmienić jedną linijkę.

```
# TODO: utwórz trzy konfiguracje (po jednej zmianie każda)
k2 = wariant("wentylator_k1.fcl", "wentylator_k2.fcl", ACT="PROD")
k3 = wariant("wentylator_k1.fcl", "wentylator_k3.fcl", METHOD="MM")
k4 = wariant("wentylator_k1.fcl", "wentylator_k4.fcl", ACCU="BSUM")

KONFIGURACJE = ["wentylator_k1.fcl", k2,k3,k4]   # TODO: dopisz k3 i k4
tabela(KONFIGURACJE, "temperature", TEMPERATURY + [28, 32, 34])
krzywa(KONFIGURACJE, "temperature")
```

```
Zapisano wentylator_k2.fcl: ACT = PROD
Zapisano wentylator_k3.fcl: METHOD = MM
Zapisano wentylator_k4.fcl: ACCU = BSUM
```

temperature	wentylator_k1	wentylator_k2	wentylator_k3	wentylator_k4
0	17.03	17.03	10.00	17.03
5	17.03	17.03	10.00	17.03
10	17.03	17.03	10.00	17.03
15	34.44	31.08	30.93	34.50
20	50.00	50.00	50.00	50.00
25	61.15	61.54	62.23	61.60
30	76.67	76.67	80.00	76.67
35	79.87	79.05	80.00	80.53
40	81.17	81.17	90.00	81.70
28	68.45	70.79	79.00	68.08
32	77.90	77.61	80.00	78.65
34	79.29	78.57	80.00	80.01



Zadanie 3A (opis). Dla każdej konfiguracji opisz: co zmieniłeś, przy jakich temperaturach wynik się zmienił i o ile. Która zmiana ma największy wpływ?

Twoja odpowiedź:

W przypadku konfiguracji k2 zmieniłam metodę operatora na iloczyn, przez co wynik zmienił się przy temperaturze 15°C (spadł o 3,36), 25°C (wzrost o 0,39), 28°C (wzrost o 2,34), 32°C (spadek o 0,29), 34°C (spadek o 0,72) oraz 35°C (spadek o 0,82). Przy konfiguracji k3 zmieniłam metodę na mean of maximum, przez co zaszedł spadek przy temperaturach 0°C, 5°C, 10°C (wszędzie o 7,03) i 15°C (o 3,51), a także wzrost przy 25°C (o 1,08), 28°C (o 10,55), 30°C (o 3,33), 32°C (o 2,10), 34°C (o 0,71), 35°C (o 0,13), 40°C (o 8,83). W przypadku konfiguracji k4 zmieniłam metodę na ograniczoną sumę, przez co wynik spadł przy 28°C (o 0,37), a tzkę wzrósł przy 15°C (o 0,06), 25°C (o 0,45), 32°C (o 0,75), 34°C (o 0,72), 35°C (o 0,66) i 40°C (o 0,53). Największy wpływ na działania sterownika miała zmiana na MM (mean of maximum) w przypadku k3.

5. Ocena sterownika

Zadanie 3B. Jak oceniasz działanie sterownika? Odnieś się do:

- czy prędkość rośnie z temperaturą płynnie, czy skokami,
- czy zakres prędkości (najmniejsza i największa) jest sensowny dla wentylatora,
- którą konfigurację wybrałbyś do prawdziwego urządzenia i dlaczego.

Twoja odpowiedź: Uważam, że prędkość pracy wentylatora rośnie płynnie wraz z temperaturą. Mimo wszystko zauważalne są w niektórych miejscach pewnie skoki w prędkości, szczególnie przy zmianie stopnia przynależności do poszczególnych zbiorów. Zakres prędkości uważam za sensowny, gdyż przy niskiej temperaturze wentylator nie wyłącza się, a wraz ze wzrostem temperatury zwiększa swoją pracę, nie osiagając tym samym 100%. Uważam, że najlepszą konfiguracją do wyboru byłaby k1, gdyż charakteryzuje się ona dośc dobrą płynnością w przejściach pracy wentylatora, który nie osiaga tzw. "skoków" między procentami pracy.

---

6. Drugie wejście: wilgotność

Przy wysokiej wilgotności to samo ciepło jest bardziej uciążliwe, więc wentylator powinien kręcić się szybciej. Dodajemy do Twojego sterownika z części 2 drugie wejście `humidity` (wilgotność powietrza, 0–100%).

6.1 Rozszerzenie pliku

Nie musisz przepisywać całego pliku. Funkcja `rozszerz` bierze `wentylator_k1.fcl` i wkleja we właściwe miejsca trzy fragmenty, które piszesz w FCL:

1. nowe wejście w `VAR_INPUT` (gotowe),
2. blok `FUZZIFY humidity` z trzema stanami: `dry`, `normal`, `humid`,
3. co najmniej dwie nowe reguły, np. „ciepło **i** wilgotno ⟶ szybko”, „gorąco **i** wilgotno ⟶ maksimum”.

```
NOWE_WEJSCIE = "humidity : REAL;     (* wilgotność [%] *)"

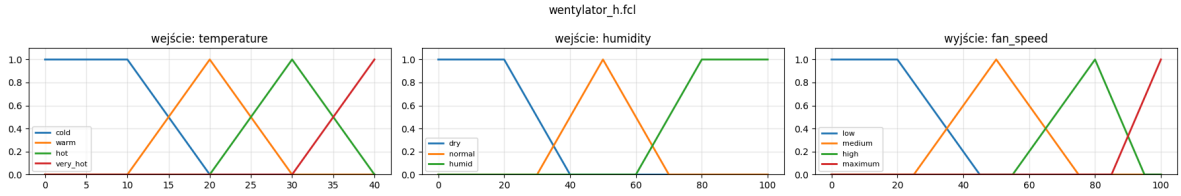
# TODO: uzupełnij punkty (x, μ) – sąsiednie stany muszą się nakładać
NOWY_FUZZIFY = """
FUZZIFY humidity
    RANGE := (0 .. 100);
    TERM dry    := (0, 1) (20, 1) (40, 0);
    TERM normal := (30, 0) (50, 1) (70, 0);
    TERM humid  := (60, 0) (80, 1) (100, 1);
```

```
END_FUZZIFY
"""

# TODO: dopisz co najmniej dwie reguły z humidity (numeruj dalej: RULE 5, RU
NOWE_REGULY = """
RULE 5 : IF temperature IS warm AND humidity IS humid THEN fan_speed IS high
RULE 6 : IF temperature IS warm AND humidity IS normal THEN fan_speed IS med
RULE 7 : IF temperature IS hot AND humidity IS humid THEN fan_speed IS maxim
RULE 8 : IF temperature IS warm AND humidity IS dry THEN fan_speed IS low;
"""

rozszerz("wentylator_k1.fcl", "wentylator_h.fcl",
         wejscie=NOWE_WEJSCIE, fuzzify=NOWY_FUZZIFY, reguly=NOWE_REGULY)
rysuj_zbiory("wentylator_h.fcl")
```

```
Zapisano wentylator_h.fcl – plik wczytuje się poprawnie.
```



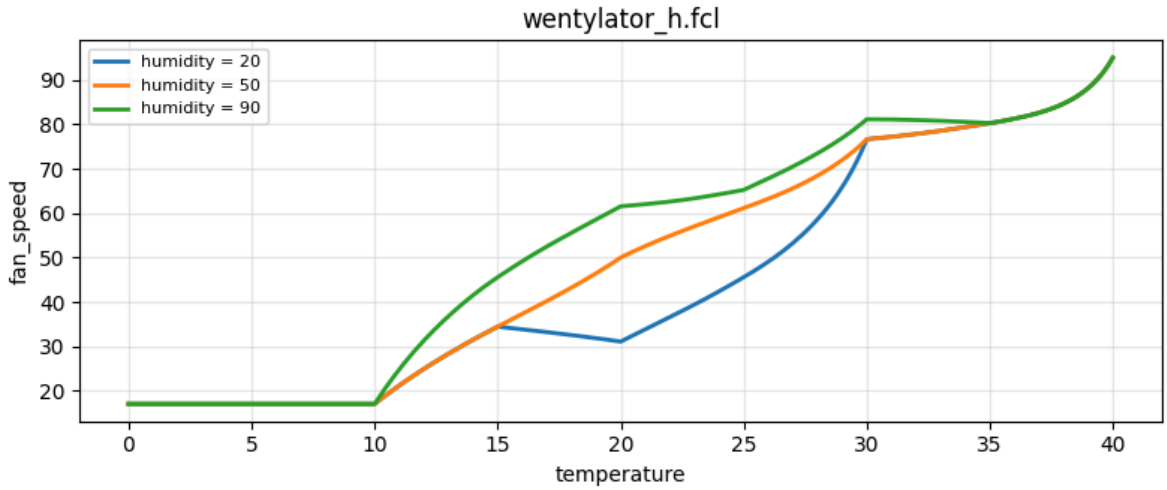
Testy. Ta sama temperatura przy suchym (20%) i wilgotnym (90%) powietrzu, a potem wilgotność od 0 do 100% przy stałych 25 °C.

```
print("Powietrze suche (humidity = 20):")
tabela("wentylator_h.fcl", "temperature", TEMPERATURY, humidity=20)
print("\nPowietrze wilgotne (humidity = 90):")
tabela("wentylator_h.fcl", "temperature", TEMPERATURY, humidity=90)
print("\nStała temperatura 25 °C:")
tabela("wentylator_h.fcl", "humidity", [0, 20, 40, 60, 75, 90, 100], tempera
krzywe("wentylator_h.fcl", "temperature", "humidity", [20, 50, 90])
```

```
Powietrze suche (humidity = 20):
 temperature  wentylator_h
           0         17.03
           5         17.03
          10         17.03
          15         34.44
          20         31.08
          25         45.58
          30         76.67
          35         80.27
          40         95.03

Powietrze wilgotne (humidity = 90):
 temperature  wentylator_h
           0         17.03
           5         17.03
          10         17.03
          15         45.58
          20         61.54
          25         65.26
          30         81.17
          35         80.27
          40         95.03

Stała temperatura 25 °C:
    humidity  wentylator_h
           0         45.58
          20         45.58
          40         61.15
          60         61.15
          75         65.26
          90         65.26
         100         65.26
```



Pytanie 6a. O ile wilgotne powietrze zwiększa prędkość przy 25 °C? Czy to dużo, czy mało?

Pytanie 6b. Czy wilgotność cokolwiek zmienia przy 10 °C? Dlaczego?

Pytanie 6c. Reguła 2 („warm → medium") nic nie wie o wilgotności. Co robi, gdy powietrze jest wilgotne? Jak to wpływa na wynik z pytania 6a?

Twoja odpowiedź:

6a. Przy temperaturze w przypadku wilgotności 20% prędkosc wentylatora to około 45%, a wilgotności 90% to około 65%. Jest to spory wzrost, oznaczający że wentylator zwiększa swoją prędkość przy wyższych wilgotnościach.

6b. Przy temperaturze 10°C prędkość wentylatora wynosi zawsze około 15%, gdyż przy niskiej temperaturze aktywna jest reguła odpowiedzialna za niską prędkosc wentylatora, a zmiana wilgotności nie wpływa w tym przypadku na wynik.

6c. Reguła 2 działa niezależnie od wilgotności. W przypadku, gdy temperatura wynosi 25°C, aktywuje się średnia prędkosc wentylatora, niezaleznie od tego, czy powietrze jest wilgotne czy nie.

6.2 Poprawka: stare reguły tylko dla niewilgotnego powietrza

Zmień reguły 2 i 3 tak, żeby działały tylko, gdy powietrze **nie** jest wilgotne. FCL ma do tego słowo `NOT`:

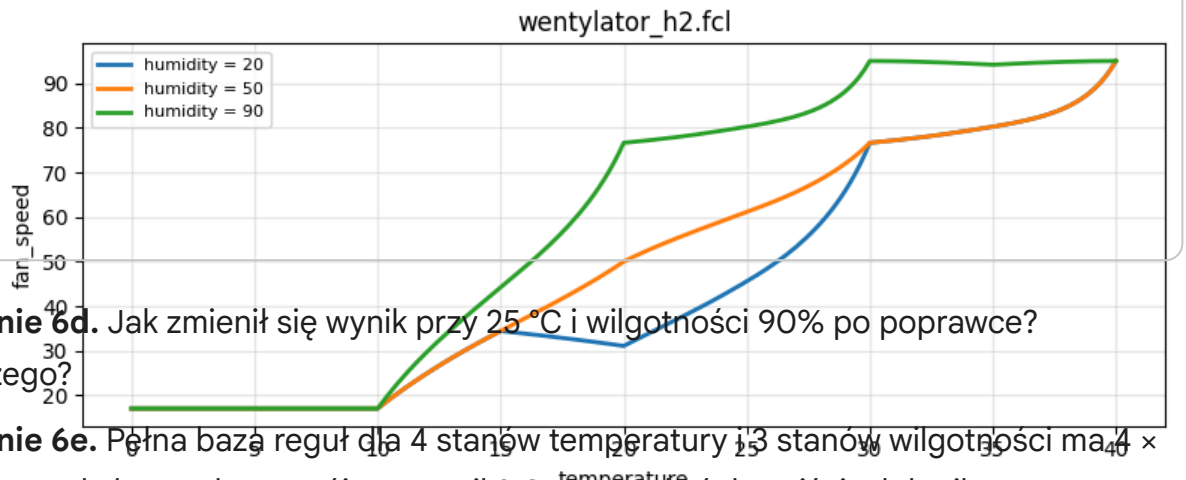
`RULE 2 : IF temperature IS warm AND humidity IS NOT humid THEN fan_speed IS medium;`

Komórka poniżej robi kopię pliku i zamienia w niej tekst reguł.

```
tekst = open("wentylator_h.fcl", encoding="utf-8").read()
# TODO: zamień reguły 2 i 3 (tekst przed zamianą musi być DOKŁADNIE taki jak
tekst = tekst.replace("RULE 2 : IF temperature IS warm THEN fan_speed IS med
                      "RULE 2 : IF temperature IS warm AND humidity IS NOT h
tekst = tekst.replace("RULE 3 : IF temperature IS hot  THEN fan_speed IS hig
                      "RULE 3 : IF temperature IS hot AND humidity IS NOT hu
open("wentylator_h2.fcl", "w", encoding="utf-8").write(tekst)

print("Stała temperatura 25 °C:")
tabela(["wentylator_h.fcl", "wentylator_h2.fcl"], "humidity", [0, 20, 40, 60
krzywe("wentylator_h2.fcl", "temperature", "humidity", [20, 50, 90])
```

```
Stała temperatura 25 °C:
    humidity  wentylator_h  wentylator_h2
           0         45.58          45.58
          20         45.58          45.58
          40         61.15          61.15
          60         61.15          61.15
          75         65.26          69.75
          90         65.26          80.27
         100         65.26          80.27
```



Pytanie 6d. Jak zmienił się wynik przy 25 °C i wilgotności 90% po poprawce? Dlaczego?

Pytanie 6e. Pełna baza reguł dla 4 stanów temperatury i 3 stanów wilgotności ma 4 × 3 = 12 reguł. Ile reguł ma Twój sterownik? Czy przy dwóch wejściach logika rozmyta daje coś, czego nie dałaby zwykła funkcja temperatury?

Twoja odpowiedź:

6d. Przy wilgotności 90% i temperaturze 25°C prędkość wentylatora wynosi około 80% zamiast 65%. Dzieje się tak przez zmianę w regule 2, która pod uwagę brała średnią prędkosc pracy wentylatora.

6e. Mój sterownik ma 8 reguł. Przy dwóch wejściach logika rozmyta pozwala na płynniejsze przechodzenie między temperaturami, unikając tym samym gwałtownych "skoków".

Do oddania